First Light PWA — Security, Correctness & Design Audit

Prepared by: Claude (Anthropic) **Scope:** `beta_v2` bundle — all HTML, CSS, JS, ES modules, service worker, manifests. Treat as static-hosting root. No Node build step.

Instructions for Cursor

You are reviewing an audit produced by another model (Claude). You have live access to the code, the deployed site, the Supabase dashboard, and runtime behaviour that the auditor did not.

For each finding:

1. **Decide if you agree.** If you disagree or can verify the concern doesn't apply in practice, say so and explain why. Don't implement fixes for issues that aren't real in your environment.
2. **Validate "needs runtime check" items first** (Section E). Many of these may already be fine — e.g., RLS may be correctly configured, Mapbox token may already be restricted. Don't waste effort on non-issues.
3. **When you agree, add the item to a to-do** with (a) the specific files to touch, (b) an estimated effort, and (c) the verify-step from the finding. Work the to-do in priority order: High → Medium → Low.
4. **One finding, one commit.** Show a unified diff (or describe the fix concretely) before applying. Run the existing tests after each fix. Don't batch multiple fixes into one commit.
5. **Preserve the positive patterns** listed in Section D. Don't refactor these without care — they encode subtle fixes the dev has already made (e.g., the SW versioning discipline, the `initialController` snapshot, the offline-queue reconcile).

If something is ambiguous, ask before implementing.

Executive summary

Overall risk: Medium. The codebase is thoughtful and well-commented; the dev has awareness of CSP, RLS, race conditions, and PWA lifecycle subtleties. Nothing here is critical. The findings cluster into five themes:

1. **One genuine privacy leak** — the error logger captures `location.href` unchanged, so password-recovery tokens (in URL hash) and syndicate-invite tokens (in query string) are logged to Supabase on any JS crash during those flows.
2. **Server-side trust boundary — needs runtime verification.** Syndicate role changes (promote/demote/remove/delete), target writes, allocation writes, and invite creation are all direct client-side Supabase writes. Defence is 100% on RLS. If RLS is anything less than strict, a non-manager member can elevate themselves, overwrite targets, or delete the syndicate.
3. **Correctness bugs** — weather null coercion returns `0.0°C` for missing data; SW install swallows all

errors and ships partial precache; 3 of 6 stats builders crash on missing DOM; EXIF orientation ignored on iOS photos.

4. **Duplication drift risk** — Supabase key, clock-sync, and `esc()` exist in two places each.
5. **Polish** — CSP missing on 4 of 7 HTML pages; iOS native `<select>` chevron stripped; 5 of 11 modals missing ARIA; manifest `privacy_policy` URL is broken; Mapbox public token shipped without URL restriction.

Strengths to preserve (see Section D): the SW versioning discipline, the escape-on-every-innerHTML discipline (~95% applied), the error logger design, the offline-queue reconcile-on-sign-in flow, and the trusted-UK-clock with proper `Intl` timezone handling.

Format of each finding

- **Severity** — Critical / High / Medium / Low / Info
- **Location** — files + line references
- **Issue** — what's wrong and why it matters
- **Suggested fix** — a specific change, not a vague direction
- **Verify** — how Cursor confirms the fix worked

A. High priority

A1. Error logger leaks auth tokens from URL on any crash [High — Privacy]

Location: `modules/error-logger.mjs` line 103. Also depends on `diary.js` lines 1553, 7934.

Issue: `row.url = clip(location.href, MAX_URL)` captures the full URL including hash and query string. Supabase password-recovery flow puts `#access_token=...&refresh_token=...&type=recovery` in the hash; syndicate-invite flow puts `?syndicate_invite=<token>` in the query. **If any JS error fires during either flow**, the token gets written to `public.app_errors`. Recovery tokens give short-term account access; invite tokens let anyone join a syndicate for up to 7 days × 10 uses.

Suggested fix: In `modules/error-logger.mjs`, sanitise `location.href` before logging:

```
function safeUrlForLog() {
  try {
    const u = new URL(location.href);
    u.hash = '';
    ['syndicate_invite','access_token','refresh_token','token','code','type','provider_token']
      .forEach(p => u.searchParams.delete(p));
    return u.toString();
  } catch (_) {
    return (typeof location !== 'undefined' ? location.pathname : null);
  }
}
```

```
// In sendError():
url: clip(safeUrlForLog(), MAX_URL),
```

Verify: Trigger a deliberate error during password recovery (e.g., throw from a breakpoint). Check `app_errors.url` column contains pathname only, no `#access_token`.

A2. Syndicate role changes / invites / deletes are unprotected client writes [High — Needs runtime check]

Location: `diary.js` lines 7797 (promote), 7824 (demote), 7858 (delete), 7874 (remove), 7673 (targets), 7688 (alloc), 7721 (invite create).

Issue: Every syndicate mutation is a direct `sb.from('...').update/delete/insert()`. There is no RPC and no client-side check that the current user is a manager — the UI merely hides the buttons. **All defence rests on Supabase RLS policies.** If RLS on `syndicate_members` / `syndicates` / `syndicate_targets` / `syndicate_member_allocations` / `syndicate_invites` is anything less than "caller must be an active manager of this syndicate_id", a non-manager opening devtools can:

- Promote themselves to manager
- Demote other managers
- Remove members
- Delete the syndicate
- Overwrite group targets
- Mint invite tokens with `max_uses: 999999` / `expires_at: 2099`

Suggested fix: This is primarily an RLS verification task, not a code change. Before anything else, confirm the following policies exist on each table:

```
-- syndicate_members: UPDATE role requires manager of same syndicate
create policy "managers update member roles" on syndicate_members
  for update using (
    exists(
      select 1 from syndicate_members m
      where m.syndicate_id = syndicate_members.syndicate_id
        and m.user_id = auth.uid()
        and m.role = 'manager'
        and m.status = 'active'
    )
  );

-- syndicates: DELETE requires manager
create policy "managers delete syndicate" on syndicates
  for delete using (
    exists(
      select 1 from syndicate_members m
      where m.syndicate_id = syndicates.id
```

```
        and m.user_id = auth.uid()
        and m.role = 'manager'
        and m.status = 'active'
    )
  );

-- syndicate_invites: INSERT requires manager, and server-side cap on max_uses/expires_at
-- Consider moving to a create_syndicate_invite RPC that enforces max_uses <= 50 and expires_at
```

If RLS is already correct, **no code change needed**, but add a comment at each call site pointing to the policy it relies on:

```
// Row-level security: only active managers of this syndicate can UPDATE role.
// See scripts/rls-syndicate-members.sql policy "managers update member roles".
var r = await sb.from('syndicate_members').update({ role: 'manager' })...
```

Additionally, move invite creation to an RPC so `max_uses` and `expires_at` are server-enforced:

```
create function public.create_syndicate_invite(p_syndicate_id uuid)
returns uuid language plpgsql security definer as $$
declare
  new_id uuid;
begin
  -- check caller is manager
  if not exists(select 1 from syndicate_members where syndicate_id = p_syndicate_id and user_id
    raise exception 'not authorized';
  end if;
  insert into syndicate_invites (syndicate_id, token, created_by, expires_at, max_uses, used_cou
    values (p_syndicate_id, encode(gen_random_bytes(24),'hex'), auth.uid(), now() + interval '7
    returning id into new_id;
  return new_id;
end$$;
```

Then replace `syndGenerateInvite` 's direct INSERT with an RPC call.

Verify: Sign in as a non-manager member. Open devtools. Run `await window.supabase.createClient(...).from('syndicate_members').update({role:'manager'}).eq('syndicate_id', '<id>').eq('user_id', '<your-id>')` . Expect: RLS rejection error. If it succeeds, RLS is broken.

A3. Weather null coercion stores missing readings as 0.0°C [High — Data correctness]

Location: `modules/weather.mjs` lines 182–192.

Issue: Open-Meteo returns `null` for missing hourly samples (documented API behaviour). The current code does `h.temperature_2m ? Math.round(h.temperature_2m[idx] * 10) / 10 : null` — but the ternary only checks if the *array* exists, not the *value at that index*. When the value is `null` , `null * 10 === 0` , so `Math.round(0) / 10 === 0` . **A missing reading is stored as `0.0` .** Same bug affects `wind_mph` , `gust_mph` , `pressure` , `cloud` , `code` , `precip_mm` .

Suggested fix:

```
function num(arr, i) { return (arr && arr[i] != null) ? arr[i] : null; }
// ...
var t   = num(h.temperature_2m, idx);
var w   = num(h.wind_speed_10m, idx);
var g   = num(h.windgusts_10m, idx);
var wd  = num(h.wind_direction_10m, idx);
var p   = num(h.surface_pressure, idx);
var c   = num(h.cloud_cover, idx);
var wc  = num(h.weather_code, idx);
var pr  = num(h.precipitation, idx);

return {
  temp:      t  != null ? Math.round(t * 10) / 10 : null,
  wind_mph:  w  != null ? Math.round(w * 0.621) : null,
  gust_mph:  g  != null ? Math.round(g * 0.621) : null,
  wind_dir:  wd,
  pressure:  p  != null ? Math.round(p) : null,
  cloud:     c,
  code:      wc,
  precip_mm: pr,
  fetched_at: diaryNow().toISOString()
};
```

Verify: Unit test with a mock Open-Meteo response where `temperature_2m[5] = null`. Expect returned `temp === null`, not `0`.

Also: Write a one-off SQL query to find and optionally patch existing bad rows: `select id from cull_entries where weather_data->>'temp' = '0' and ...` — decide whether to null them or leave them.

A4. Service worker install swallows all errors, may ship partial precache [High]

Location: `sw.js` lines 160–173.

Issue: `Promise.all(allUrls.map(url => cache.add(url).catch(e => console.warn(...))))` means any failed precache URL is silently swallowed. Combined with `self.skipWaiting()` at line 170, a broken release activates instantly on every open tab with a partially-populated cache — users are told they're offline-ready when they're not. This is doubly risky because same-origin precache URLs are the app's own files: a typo in `PRECACHE_URLS` ships to production undetected.

Suggested fix: Split same-origin (atomic) from CDN (best-effort), and gate `skipWaiting` on client opt-in:

```
self.addEventListener('install', event => {
  event.waitUntil((async () => {
    const cache = await caches.open(STATIC_CACHE);
    // Same-origin: fail fast if any is broken — these are OUR files
    await cache.addAll(PRECACHE_URLS);
    // CDN: best-effort, don't abort install if one CDN is unreachable
    await Promise.all(CDN_URLS.map(u =>
      cache.add(u).catch(e => console.warn('[SW] CDN precache skipped:', u, e))
    ));
  })());
```

```
});
self.addEventListener('message', event => {
  if (event.data && event.data.type === 'SKIP_WAITING') self.skipWaiting();
});
```

Then in `modules/sw-bridge.mjs`, update the Reload button handler:

```
btn.addEventListener('click', function() {
  if (navigator.serviceWorker.controller) {
    // Tell the waiting SW to activate, then reload
    navigator.serviceWorker.ready.then(reg => {
      if (reg.waiting) reg.waiting.postMessage({ type: 'SKIP_WAITING' });
    });
  }
  location.reload();
});
```

Verify: Add a bogus URL to `PRECACHE_URLS` (`'./does-not-exist.js'`), bump `SW_VERSION`, deploy to a test branch. Expect the SW fails to install (visible in DevTools → Application → Service Workers). Remove the bogus URL, bump again, confirm install succeeds.

A5. Four HTML pages ship without any CSP [High]

Location: `deerschool.html`, `privacy.html`, `terms.html`, `diary-guide.html`.

Issue: Only `index.html` and `diary.html` have a CSP meta tag. `deerschool.html` is dynamic (JS-driven quiz with 14 `innerHTML` sinks). `privacy.html` and `terms.html` are static but still served from the same origin as authenticated Supabase sessions — an injection on those pages has origin privileges. `diary-guide.html` is linked from the auth card.

Suggested fix: For the **static docs** (privacy.html, terms.html), use a maximally-strict CSP:

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'none'; style
```

For `deerschool.html`, copy `index.html`'s CSP (it's already tight — no unpkg, no mapbox). Adjust if deerschool actually needs connect-src entries.

For `diary-guide.html`, use the static-docs CSP (it's informational only).

Verify: Load each page, open DevTools → Network → Headers (or check the `<meta>` tag), confirm `Content-Security-Policy` is present. Try pasting `<img src=x onerror=alert(1)>` into the URL fragment or localStorage if a path exists — expect CSP to block.

A6. Mapbox public token shipped without URL restriction [High — Needs runtime check]

Location: `diary.html` line 18 (`<meta name="fl-mapbox-token" content="pk.ey...">`)

Issue: Public `pk.*` tokens are *designed* to ship to browsers, but if URL restrictions aren't configured in the Mapbox dashboard, anyone can lift the token and use it against your quota / billing.

Suggested fix: No code change. In the Mapbox account dashboard, edit the token and restrict allowed referrers to `https://firstlightdeer.co.uk/*` and `http://localhost:*` . Consider a second dev-only token scoped to localhost.

Verify: In an incognito window on a domain you control (e.g., a CodePen), try using the token against the Mapbox API. Expect 403/401 once restriction is applied.

A7. Three stats card-builders will crash if DOM nodes are missing [High]

Location: `modules/stats.mjs` lines 241–247 (`buildShooterStats`), 269–273 (`buildDestinationStats`), the card lookups in `buildCalibreDistanceStats` (line 355, 391), and the start of `buildAgeStats` .

Issue: `buildTimeOfDayStats` (line 310), `buildTrendsChart` (line 573), and `buildGroundStats` (line 625) correctly do `if (!card || !chart) return;` . The other three don't. If the DOM ever doesn't match — stale cached HTML, race with view mounting, future refactor — those throw `TypeError: Cannot read properties of null (reading 'style')` , and because `renderStatsTabBody` calls them in sequence without try/catch, the whole stats tab blanks.

Suggested fix: Add the guard to each. Example for `buildShooterStats` :

```
export function buildShooterStats(entries) {
  var card  = document.getElementById('shooter-card');
  var chart = document.getElementById('shooter-chart');
  if (!card || !chart) return;                          // ← add
  var agg = aggregateShooterStats(entries);
  // ... rest unchanged
}
```

Same pattern for `buildDestinationStats` , `buildAgeStats` , and the two card sections inside `buildCalibreDistanceStats` .

Verify: Temporarily remove `<div id="shooter-card">` from `diary.html` , load stats, expect no console error and other cards render normally.

B. Medium priority

B1. Supabase session stored in localStorage; refresh token exposed to any XSS

Location: `modules/supabase.mjs` line 67.

Issue: `createClient(URL, KEY)` uses Supabase's default `localStorage` -based session storage. Refresh token lives under `sb-sjaasuqeknvvmdpydfsz-auth-token` and is stealable by any successful XSS. Current CSP is strong, but `style-src 'unsafe-inline'` and ~108 `innerHTML` sinks keep the exposure non-zero.

Suggested fix: Consider session-scoped storage:

```
sb = window.supabase.createClient(SUPABASE_URL, SUPABASE_KEY, {
  auth: {
```

```
    storage: window.sessionStorage,   // or custom in-memory with refresh fallback
    persistSession: true,
    autoRefreshToken: true,
    detectSessionInUrl: true
  }
});
```

Trade-off: users re-sign-in on every browser restart. That may be unacceptable UX — if so, document the threat model explicitly and move on.

Verify: After fix, close and reopen browser, expect sign-in prompt.

B2. EXIF orientation ignored — iOS camera photos render sideways

Location: `modules/photos.mjs` `compressPhotoFile` lines 130–173.

Issue: `ctx.drawImage(img, 0, 0, w, h)` doesn't honour EXIF `Orientation` tag. iPhone photos taken in portrait often have `Orientation: 6` (rotate 90° CW); the canvas output comes out sideways. iOS Safari renders `<img>` EXIF-correctly but canvas draws ignore it unless you use `createImageBitmap(file, { imageOrientation: 'from-image' })`.

Also: No input bounds. A 10,000×10,000 image decoded on a low-end phone can OOM-crash Safari.

Suggested fix:

```
export function compressPhotoFile(file, opts) {
  opts = opts || {};
  var maxDim   = opts.maxDim   || 800;
  var quality  = (opts.quality != null) ? opts.quality : 0.75;
  var mimeType = opts.mimeType || 'image/jpeg';
  var filename = opts.filename || 'photo.jpg';

  // Reject oversized inputs before decode
  if (file.size > 20 * 1024 * 1024) {
    return Promise.reject(new Error('Photo too large (max 20 MB)'));
  }

  // Prefer createImageBitmap path (EXIF-aware on iOS 16+, most Android)
  if (typeof createImageBitmap === 'function') {
    return createImageBitmap(file, { imageOrientation: 'from-image' })
      .then(function(bitmap) {
        var w = bitmap.width, h = bitmap.height;
        if (w * h > 25 * 1000 * 1000) {
          bitmap.close && bitmap.close();
          throw new Error('Photo resolution too high (max ~25 megapixels)');
        }
        if (w > h) { if (w > maxDim) { h = Math.round(h * maxDim / w); w = maxDim; } }
        else { if (h > maxDim) { w = Math.round(w * maxDim / h); h = maxDim; } }
        var canvas = document.createElement('canvas');
        canvas.width = w; canvas.height = h;
        canvas.getContext('2d').drawImage(bitmap, 0, 0, w, h);
        bitmap.close && bitmap.close();
```

```
        return new Promise(function(resolve, reject) {
          canvas.toBlob(function(blob) {
            if (!blob) return reject(new Error('Canvas toBlob returned null'));
            var f = new File([blob], filename, { type: mimeType });
            resolve({ blob: blob, file: f, previewUrl: URL.createObjectURL(blob), kb: Math.round
          }, mimeType, quality);
        });
      });
  }

  // Fallback: legacy FileReader path (no EXIF awareness, but some iOS versions lack createImage
  // ... existing code ...
}
```

Verify: On iPhone, photograph something with a recognisable “up” direction. Upload. Check the thumbnail and PDF render orientation-correctly.

B3. Three places store the Supabase anon key

Location: `modules/supabase.mjs` lines 36–37 and `app.js` lines 45–46.

Issue: Rotation requires updating both. One will drift.

Suggested fix: Have `app.js` dynamically import from the module:

```
// app.js — early init
import('./modules/supabase.mjs').then(m => {
  window.FL_SUPABASE_URL = m.SUPABASE_URL;
  window.FL_SUPABASE_KEY = m.SUPABASE_KEY;
  // defer anything that needs these
});
```

Or if `app.js` must stay a classic script, use a generated include file (`config.js`) that both consume.

Verify: `grep -r "eyJhbGciOiJIUzI1NiI..." beta_v2/` should return one hit.

B4. Clock-sync implementation duplicated between `app.js` and `modules/clock.mjs`

Location: `app.js` lines 38–116; `modules/clock.mjs` .

Issue: `modules/clock.mjs` comment on line 20 says “byte-for-byte identical”. Today maybe. Tomorrow one will drift.

Suggested fix: Dynamic import of `modules/clock.mjs` from `app.js` , delete the inline duplicate. Both use the same localStorage keys already, so no user-visible effect.

Verify: Delete `app.js` lines 41–116. Confirm Field Guide still shows correct legal hours (uses the synced offset).

B5. `esc()` defined in two places

Location: `lib/fl-pure.mjs` line 182; `diary.js` line 2642.

Issue: Identical bodies. Drift risk.

Suggested fix: In `diary.js`, add `esc` to the import list from `lib/fl-pure.mjs`, delete the local definition.

Verify: `grep -n "function esc" beta_v2/` returns one hit.

B6. Two PWA manifests with identical icons — confuses users on iOS home screen

Location: `manifest.json`, `manifest-diary.json`.

Issue: Two installable PWAs ("First Light", "Cull Diary") sharing the same icon set — indistinguishable on home screen. Also: is dual-install even desired? iOS users expect one app per service.

Suggested fix: Either **(a)** differentiate — generate a second icon set for Cull Diary with a notebook/checklist glyph overlay on a matching base, point `manifest-diary.json` to those; or **(b)** collapse to one installable PWA with Cull Diary as a [manifest shortcut](#):

```
"shortcuts": [
  { "name": "Cull Diary", "short_name": "Diary", "url": "./diary.html", "icons": [{"src":"icon-1
]
```

Recommend (b) unless users have asked for a dedicated diary icon.

Verify: Install to iOS home screen from Safari, confirm one icon (b) or visually distinct icons (a).

B7. `manifest.json` `privacy_policy` URL returns 404

Location: `manifest.json` line 20, `manifest-diary.json` line 20.

Issue: Points to `https://firstlightdeer.co.uk/privacy` — file is `privacy.html`. On flat GitHub Pages, `/privacy` is a 404.

Suggested fix:

```
"privacy_policy": "https://firstlightdeer.co.uk/privacy.html"
```

Or add a GitHub Pages redirect if "pretty URLs" are desired project-wide.

Verify: `curl -I https://firstlightdeer.co.uk/privacy` returns 200 after fix (either by changing the manifest or adding a redirect).

B8. iOS native `<select>` in stats header has no visible affordance

Location: `diary.css` line 194 (`.season-pill-sel`), `diary.html` line 494.

Issue: `-webkit-appearance:none` strips the native dropdown chevron on iOS, so users see a pill with no indication it's tappable. Also `max-width:min(230px,48vw)` truncates long season labels on narrow screens.

Suggested fix: Restore a CSS-drawn chevron and add ellipsis:

```
.season-pill-sel{
  background: rgba(200,168,75,0.12)
    url("data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='10' height='6' v
  padding: 5px 26px 5px 10px;
  text-overflow: ellipsis;
  white-space: nowrap;
  overflow: hidden;
  /* ... keep the rest ... */
}
```

Verify: On iOS Safari, open stats tab, confirm chevron visible and long season labels ellipsise instead of overflowing.

B9. Five modals missing ARIA

Location: `diary.html` lines 68 (forgot-password), 739 (delete-account), 827 (export), 858 (syndicate-export), 1118 (summary-filter).

Issue: Screen-reader users aren't notified when these dialogs open. Includes two destructive/sensitive ones (delete account, forgot password).

Suggested fix: Add the ARIA triplet to each:

```
<div id="forgot-password-modal" class="..." style="display:none;"
     role="dialog" aria-modal="true" aria-labelledby="forgot-password-title">
  <div class="di-modal-card">
    <div class="di-modal-title" id="forgot-password-title">Reset password</div>
    <!-- ... -->
```

Apply the same pattern to the other four. If a modal has no heading element, add one or use `aria-label="..."` instead.

Verify: Open each with VoiceOver (iOS) or NVDA (desktop), confirm the dialog title is announced.

B10. PDF filenames interpolate DB fields without sanitation

Location: `modules/pdf.mjs` lines 611–614, 729, 1302.

Issue: `entry.species` / `entry.date` / `entry.time` / `entries[0].date` go into `doc.save(filename)` with only `\s+` replacement. Currently enum-safe, but `syndicateFileSlug()` at line 88 already has the right pattern (`[^a-z0-9]+`) — it's just not applied everywhere.

Suggested fix: Make a small private helper and use it on every filename piece:

```
function fileSlug(s) {
  return String(s || '').replace(/[^a-z0-9-]/gi, '-').replace(/^-|-$/g, '').toLowerCase() || 'x'
}
// Then:
var speciesSlug = fileSlug(entry.species);
var datePart    = fileSlug(entry.date);
var timeSlug    = fileSlug(fmtEntryTimeShort(entry.time));
// ... etc
```

Verify: Set an entry's species to `Red/../Deer` (via SQL if needed), export PDF, confirm filename is `cull-record-red-deer-2024-01-01.pdf`, not `cull-record-red/../deer-...`.

B11. IndexedDB photo IDs not user-scoped (edge-case data-mixing on shared devices)

Location: `diary.js` line 6001.

Issue: `_id = Date.now() + '-' + rand(5)`. On a shared device with two users A and B, a vanishingly unlikely ID collision between A's queued photo and B's new photo could result in B's sync uploading A's photo. The namespace is shared across users.

Suggested fix: Prefix photo IDs with the user ID:

```
entry._queued_user_id = getCurrentQueueUserId();
entry._id = entry._queued_user_id + '-' + Date.now() + '-' + Math.random().toString(36).slice(2,
```

And in cleanup paths, also clear IndexedDB photos on sign-out that aren't referenced by any remaining queue item for any user.

Verify: Manual test — sign in as user A, queue a photo offline, sign out, sign in as user B, expect no leaked blob visible.

B12. Syndicate RPC error echoes raw Postgres message to toast

Location: `diary.js` line 7946 (also lines 7815, 7867, etc.).

Issue: `'⚠️ Invite: ' + (e.message || 'invalid')` shows users strings like "duplicate key value violates unique constraint 'syndicate_invites_token_key'" — XSS-safe (toast uses `textContent`) but unpolished and leaks schema detail.

Suggested fix: Map known errors to user-friendly messages; log the raw message instead of showing it:

```
function friendlyRpcError(e) {
  var m = e && e.message ? String(e.message) : '';
  if (/already a member/i.test(m)) return 'You're already a member of this syndicate.';
  if (/expired|not found/i.test(m)) return 'This invite has expired or is no longer valid.';
  if (/max_uses|use_count/i.test(m)) return 'This invite has reached its use limit.';
  return 'Invite could not be redeemed.';
}
// ...
```

```
showToast('⚠️ ' + friendlyRpcError(e));
console.warn('redeem_syndicate_invite error:', e);
```

Verify: Trigger each error path with a known-bad token, confirm toast shows the friendly message.

B13. Invite tokens in query string land in Referer / server logs

Location: `diary.js` line 6956 (`syndicateInviteUrl`).

Issue: `?syndicate_invite=<token>` is in the query string. Query strings appear in `Referer` headers when users click any outbound link from the app page, get logged by web servers, and may be captured by analytics. 7-day, 10-use tokens are long-lived enough for this to matter. See also A1 (error logger).

Suggested fix: Move the token to the URL fragment (not sent to servers, not in Referer):

```
function syndicateInviteUrl(token) {
  return window.location.origin + window.location.pathname + '#syndicate_invite=' + encodeURICom
}
```

And update `tryRedeemSyndicateInviteFromUrl` to read from `location.hash` :

```
async function tryRedeemSyndicateInviteFromUrl() {
  if (!sb || !currentUser) return;
  var tok = null;
  // New: hash
  var h = (window.location.hash || '').replace(/^#/, '');
  if (h) {
    var hp = new URLSearchParams(h);
    tok = hp.get('syndicate_invite');
  }
  // Back-compat: also read from query (old invites still in the wild)
  if (!tok) {
    var sp = new URLSearchParams(window.location.search);
    tok = sp.get('syndicate_invite');
  }
  if (!tok) return;
  // ... rest unchanged
}
```

Verify: Generate new invite, share the URL. Load it. Confirm redemption works. Check Referer from any outbound link on the app page — expect no token.

B14. Syndicate name has no length or content validation

Location: `diary.js` line 7434.

Issue: Client only checks `!name` . 1MB name creates issues; profane or spammy names visible to all members. Server-side check preferable; client adds a UX guardrail.

Suggested fix: Client-side limit (server-side should also constrain):

```
name = name.trim();
if (!name) { showToast('⚠️ Enter a name'); return; }
if (name.length > 60) { showToast('⚠️ Name too long (max 60 characters)'); return; }
```

And add a DB constraint: `alter table syndicates add constraint name_length check (char_length(name) between 1 and 60);`

Verify: Try to save a 100-char name. Expect rejection.

C. Low priority

C1. `diary.js` has a second copy of `esc()`

Already covered in B5. (Listed again for completeness of the action list.)

C2. `enhanceKeyboardClickables` is effectively dead code

Location: `deerschool.js` lines 20–38; also called from `diary.js`. **Issue:** Scans for `[onclick]` elements. No HTML in the bundle uses `onclick`. The function runs but matches nothing. **Suggested fix:** Either extend it to handle `[data-fl-action]` elements (the real interactive pattern), or delete it. **Verify:** `grep -rn 'onclick=' beta_v2/` returns 0 hits in HTML and 0 hits in JS-built HTML.

C3. Inconsistent line endings across the repo

Location: `modules/*.mjs` use CRLF; `diary.js`, `app.js`, HTML files use LF. **Suggested fix:** Add `.gitattributes`:

```
* text=auto eol=lf
```

Then `git add --renormalize .` and commit. **Verify:** `file beta_v2/modules/*.mjs` reports LF.

C4. `window._activeTab` should be `ui._activeTab`

Location: `app.js` line 1628.

C5. Stats module has one unescaped species name

Location: `modules/stats.mjs` line 542 (`'<div class="age-sp-nm">'+sp+'</div>'`). **Fix:** `esc(sp)`.

C6. `'__untagged__'` sentinel collision

Location: `modules/stats.mjs` line 631. **Fix:** Use a sentinel like `'\u0000__untagged__'` or, better, a Symbol kept in a side Map.

C7. CSS selector built from localStorage without validation

Location: `app.js` line 1621. **Fix:** Validate `tab` against `Object.keys(tabMap)` before using in a selector.

C8. Weather URL built via string concat

Location: `modules/weather.mjs` lines 165–168. **Fix:** Use `URL` + `searchParams.set()` with `Number(lat)` / `Number(lng)` coercion.

C9. Numeric fields interpolated into HTML without coercion

Location: ~10 sites in `diary.js` (e.g., `e.weight_kg + ' kg'`). **Fix:** Wrap in `Number(x)` or `esc(String(x))` as belt-and-braces.

C10. Opportunistic CDN caching has no path allow-list

Location: `sw.js` `CACHEABLE_ORIGINS` lines 80–86. **Fix:** Tighten `isCacheableCDN` to exact package paths, e.g., only cache `unpkg.com/leaflet.markercluster@1.5.3/...`.

C11. jsDelivr + unpkg scripts don't use SRI

Location: `diary.html` (markercluster and supabase-js). **Fix:** Pin versions and add `integrity="sha384-..."` hashes matching your current snapshot.

C12. 14 font files across both pages

Location: `index.html` and `diary.html` load different Google Fonts URLs. **Fix:** Consolidate to one shared URL for cache warming, drop unused weights.

C13. `'unsafe-inline'` in `style-src` across both CSPs

Location: `diary.html`, `index.html` CSPs. **Fix:** Long-term — migrate inline `style="..."` accumulation in `diary.js` to CSS classes + CSS custom properties, drop `'unsafe-inline'` from `style-src`. Big refactor, not urgent.

C14. `diary.css` is 106 KB

Probably 15–25% dead or foldable. Low priority; schedule after content changes settle.

C15. Supabase `@2` precached without pinning

Location: `sw.js` line 69. **Fix:** Pin to current minor version (`@2.45.4` or whatever is current), accept that bumping requires an `SW_VERSION` bump.

C16. Console warnings in production

Location: `modules/weather.mjs` line 194, `modules/stats.mjs` and RPC error handlers. **Fix:** Gate with a debug flag (`if (window.__flDebug) console.warn(...)`) or accept them.

D. Positive notes (preserve these patterns)

- **`sw.js` SW_VERSION unified source.** The comment on lines 2–8 shows awareness of the prior drift bug.
- **`modules/sw-bridge.mjs` `initialController` snapshot** prevents first-install update-banner flash — a subtle bug most PWAs have.
- **`modules/error-logger.mjs`** dedupe + session cap + never-recurse + clip-length + pointer to SQL

schema. Excellent.

- **`modules/photos.mjs` `newCullPhotoPath`** handles same-ms collisions with a 6-char random suffix; comment explains why.
- **`modules/weather.mjs` `diaryLondonWallMs`** correctly handles BST↔GMT transitions AND user-travelling-abroad via `Intl.DateTimeFormat` with `longOffset`.
- **`modules/supabase.mjs`** module live-binding pattern (`export let sb = null; ... sb = createClient(...)`) lets consumers see the post-init value cleanly.
- **Nominatim handling in `app.js` lines 3371–3393** uses `createElement` + `textContent` — explicit comment "DOM creation, no onclick= string" shows CSP-awareness.
- **`modules/pdf.mjs`** builds PDFs entirely via `doc.text()` / `doc.rect()` / `doc.line()` — no `doc.html()` / `fromHTML()` / `innerHTML`. Injection-safe by construction.
- **`modules/stats.mjs` `_setText` / `_setHtml` helpers** at lines 734–735 show the correct null-guarded-DOM-write pattern. The three unguarded builders are outliers, not the rule.
- **Anon JWT verified** (`role: 'anon'`, `exp: 2036-04-11`). No `service_role` key in client code.
- **`diary.js` offline queue `reconcileOfflineQueueForCurrentUser`** (line 5865) actively scopes the queue to the current user on sign-in and toasts the user if it cleared another account's entries. Well-handled.
- **Syndicate code acknowledges the RLS model** in comments at lines 7127–7128. Architecture is correct *if* RLS is correct.
- **CSP on `index.html`** is tighter than `diary.html` (no unpkg, no mapbox) — appropriate for Field Guide which doesn't need maps.
- **Safe-area handling** (`env(safe-area-inset-bottom, fallback)`) consistent across sheets, modals, scroll containers.
- **24-byte `crypto.getRandomValues` invite token** (192-bit entropy). Correct choice.

E. Items that need runtime verification (Cursor: validate first)

These are the items the auditor could not verify from the bundle alone. Validate these before implementing fixes, because they may already be fine:

1. **RLS policies on `syndicate_members`, `syndicates`, `syndicate_targets`, `syndicate_member_allocations`, `syndicate_invites`** — A2. Run the devtools "promote yourself" test as a non-manager.
2. **Mapbox token URL restriction** — A6. Test in an incognito/external context.
3. **`app_errors` insert policy scope** — can any authenticated user insert rows with `user_id != auth.uid()` ? Should be restricted to self-writes.
4. **`privacy_policy` URL resolution** — B7. `curl` it.

5. **Stats-header `<select>` on iOS Safari** — B8. Physical device test.
6. **Keyboard activation on `role="button"` cards** — ensure Enter/Space triggers `data-fl-action` dispatch.
7. **Focus trap + initial focus in modals** — on open, first focusable should receive focus; Tab should cycle within modal.
8. **Gold-on-dark (`#c8a84b` on dark backgrounds) contrast** — WCAG AA needs 4.5:1 for small text. Check with a contrast tool for the 12px "How to use the Cull Diary" link on the auth card.
9. **Hit targets on small buttons** — forgot-password link, stats filter chips — 44×44pt minimum.
10. **Open-Meteo null-response behaviour in the wild** — A3. Query `app_errors` or `cull_entries.weather_data` for rows with `temp = 0` clustered during API outages.

F. Suggested execution order (assuming Cursor agrees)

Sprint 1 — privacy + correctness (4–6 hours):

1. A1 — Error-logger sanitation
2. A3 — Weather null coercion
3. A7 — Stats DOM guards
4. A6 — Mapbox token restriction (dashboard only)
5. B7 — Manifest privacy URL

Sprint 2 — RLS & server trust (1–3 days depending on existing RLS): 6. A2 — **VERIFY RLS FIRST**, then either add comments pointing to policies, or fix the policies, or move invite creation to an RPC.

Sprint 3 — hardening (1 day): 7. A4 — SW install atomicity + gate skipWaiting 8. A5 — CSP on 4 missing pages 9. B13 — Invite tokens into hash instead of query 10. B8 — Restore `<select>` chevron 11. B9 — Missing ARIA on 5 modals 12. B12 — Friendlier RPC error messages

Sprint 4 — de-dup + mobile polish (1–2 days): 13. B3, B4, B5 — De-duplicate Supabase key, clock-sync, `esc` 14. B2 — EXIF orientation + upload bounds 15. B6 — Resolve two-manifest situation 16. B10 — PDF filename sanitisation 17. B11 — User-scope IndexedDB photo IDs 18. B14 — Syndicate name validation + DB constraint

Backlog (C items): Individual line-item fixes; mostly 5–15 min each. Batch by file.

G. Assumptions and out-of-scope

The auditor assumed:

- Bundle contents are the full deployed site
- No Node build step on the server (so modules ship as-written)

- Supabase is the only backend
- The dev has SQL access to Supabase for RLS verification and RPC creation

Out-of-scope (can't be audited from bundle):

- Supabase RLS policies
- Supabase RPC function bodies (`redeem_syndicate_invite` , `create_syndicate` , `syndicate_season_summary` , etc.)
- `app_errors` table insert policy
- Deployed CSP headers (if any are set at the web-server level in addition to meta)
- GitHub Pages redirect rules
- Mapbox token dashboard configuration
- Actual runtime behaviour on iOS Safari / Android Chrome
- Performance on low-end devices

End of report.